

Lista de Atividades

Exercício 05: Interface entre C++, ROOT e Python

Marco Dias

Contexto

No Exercício 04 vocês escreveram dois macros do ROOT independentes: um que gerava $N = 1000$ números aleatórios com distribuição normal e os armazenava em uma **TTree** dentro de um **TFile**, e um segundo que lia essa árvore de volta, plotava os dados, estilizava o histograma e fazia o ajuste (fit).

Esses macros eram interpretados diretamente pelo **root** (via CINT/Cling) e nunca chegaram a ser compilados explicitamente, e também nunca saíram do ambiente do ROOT. Em fluxos de análise reais, no entanto, é constante a necessidade de transitar código e dados *entre* linguagens: código C++ que é ligado (*linked*) contra as bibliotecas do ROOT e compilado com uma ferramenta de build, e código Python que conversa com objetos do ROOT através do PyROOT. Este exercício trata justamente dessa camada de interface.

Vocês não vão escrever física ou estatística nova – vão pegar os dois scripts que já têm e reescrevê-los em dois outros ambientes, de forma que ao final vocês tenham **três implementações equivalentes** do mesmo exercício:

- (a) macros do ROOT (já feito, Exercício 04),
- (b) programas C++ compilados que se ligam às bibliotecas do ROOT,
- (c) scripts em PyROOT.

Parte 1 – C++ e ROOT: compilando contra as bibliotecas do ROOT

Peguem os dois scripts do Exercício 04 e os transformem em dois **programas C++ independentes**, `generate.cpp` e `plot_and_fit.cpp`. Isso significa:

- Cada arquivo deve ter uma função `int main()` de verdade (macros do ROOT, por outro lado, são chamados pelo nome de uma função que coincide com o nome do arquivo e só podem ser interpretados, nunca viram um executável de verdade).
- Vocês devem incluir explicitamente (`#include`) os cabeçalhos do ROOT que usarem (por exemplo `TFile.h`, `TTree.h`, `TH1F.h`, `TRandom3.h`, `TCanvas.h`, `TF1.h`), em vez de depender do carregamento automático de dicionários do interpretador.
- A geração de números aleatórios, o preenchimento da árvore, a estilização do histograma (linha preta sólida mais grossa, preenchimento amarelo, fundo branco, rótulos nos eixos) e o ajuste devem se comportar exatamente como no Exercício 04.
- Escrevam um **Makefile** que compile os dois programas contra as bibliotecas do ROOT usando o `root-config`. Um esqueleto mínimo é o seguinte:

```

1 CXX      = g++
2 CXXFLAGS = -O2 -Wall -std=c++17 $(shell root-config --cflags)
3 LDFLAGS   = $(shell root-config --libs)
4
5 all: generate plot_and_fit
6
7 generate: generate.cpp
8     $(CXX) $(CXXFLAGS) -o $@ $< $(LDFLAGS)
9
10 plot_and_fit: plot_and_fit.cpp
11     $(CXX) $(CXXFLAGS) -o $@ $< $(LDFLAGS)
12
13 clean:
14     rm -f generate plot_and_fit *.o

```

- Ao rodar `make`, devem ser gerados dois executáveis, `./generate` e `./plot_and_fit`, sem erros nem warnings. Ao rodar `./generate`, deve ser criado o mesmo `TFile/TTree` do macro do Exercício 04, e `./plot_and_fit` deve ler esse arquivo e reproduzir o gráfico estilizado e ajustado (salvo em uma imagem, por exemplo `.png` ou `.pdf`).

Pergunta para responder no README (2–3 frases): o que `root-config -cflags` e `root-config -libs` retornam de fato na máquina de vocês, e por que o compilador precisa dessa informação quando ela não era necessária dentro do interpretador do ROOT?

Parte 2 – Python e ROOT: PyROOT

Reescrevam os mesmos dois exercícios como dois **scripts em PyROOT**, `generate.py` e `plot_and_fit.py`, executados com `python3` puro (e não `root -l`):

```

1 import ROOT
2
3 f = ROOT.TFile("data.root", "RECREATE")
4 tree = ROOT.TTree("tree", "Numeros aleatorios gaussianos")
5 ...

```

Requisitos:

- Usar o módulo `ROOT` do Python (PyROOT) para criar o `TFile`, a `TTree`, preenchê-la com $N = 1000$ números com distribuição normal e salvá-la – exatamente como no Exercício 04, mas a partir do Python.
- Em `plot_and_fit.py`, ler a árvore de volta, desenhar o histograma com a mesma estilização (linha preta sólida mais grossa, preenchimento amarelo, fundo branco do canvas, rótulos nos eixos) e ajustá-lo com `TF1`, tudo chamado a partir do Python.
- Anotar onde a sintaxe do PyROOT difere da do C++ (por exemplo, não é preciso declarar tipos, o acesso a atributos substitui o `->`, diferenças de palavras-chave), e onde é uma tradução direta, um para um.

Pergunta para responder no README (2–3 frases): o PyROOT é código Python que, no fim, chama as mesmas classes C++ do ROOT que vocês usaram na Parte 1. Como esse elo entre o Python e as bibliotecas compiladas do ROOT em C++ realmente acontece? (Não precisam explicar os detalhes internos do `cppy` – basta mostrar que entendem que o Python não está reimplementando o ROOT, e sim chamando-o.)

Parte 3 – Comparando as três versões

No `README.md` do repositório, incluam uma tabela comparativa curta (ou alguns parágrafos) discutindo:

- Qual versão foi mais rápida de *escrever*, e qual foi mais rápida de *executar*?
- O que teve que mudar no código (não só a sintaxe) ao passar de um macro interpretado do ROOT, para um programa C++ compilado e ligado ao ROOT, para um script Python que chama o ROOT via PyROOT?
- Em qual dos três casos vocês pegariam um erro de tipo antes de rodar o programa, e em qual só o pegariam em tempo de execução? Por quê?

Entregáveis

Enviem (*push*) o seguinte para o repositório **público** de vocês no GitHub, em uma pasta chamada `exercise05/`:

- `generate.cpp`, `plot_and_fit.cpp`, `Makefile`
- `generate.py`, `plot_and_fit.py`
- O(s) gráfico(s) de saída (por exemplo `fit_cpp.png`, `fit_pyroot.png`)
- `README.md` com as duas respostas curtas das Partes 1–2 e a comparação da Parte 3

Mandem o link do commit (ou do repositório, se for novo) assim que tiverem enviado os resultados.